
Extending Surrogate Hamiltonian Monte Carlo

Abhijit Brahme^{* 1}

Abstract

The advent of automatic differentiation (Baydin et al., 2018) has allowed gradient based MCMC methods to prosper. For example, Hamiltonian Monte Carlo (Brooks et al., 2011) (HMC) has been the workhorse of Bayesian inferential methods, allowing for efficient sampling from high dimensional posterior distributions. Advancements including the No U-Turn Sampler (Hoffman & Gelman, 2011) (NUTS) as well as Riemannian Manifold Hamiltonian Monte Carlo (Betancourt, 2013) (RMHMC) further improved the efficiency of the standard sampling algorithm. Because gradient calculations can be computationally expensive, “surrogate” HMC methods were developed; these methods aim to learn an approximation of a computationally complex aspect of HMC during the burn-in phase, and apply the approximation during the sampling phase in order to speed up the algorithm. Recent advancements in neural network architecture such as the development of Hamiltonian Neural Networks (Greydanus et al., 2019) (HNNs) and Neural ODEs (Chen et al., 2019) offer a natural architectural extension of these “surrogate” methods, and allow for the explicit preservation of Hamiltonian qualities. We show several possible architectural extensions to surrogate methods using these recent advancements, and illustrate their utility as compared to baseline surrogate models.

1. Introduction

1.1. Motivation

Hamiltonian Monte Carlo (HMC) and its variants are a staple of the Bayesian community, providing a robust algorithm to efficiently sample from challenging posterior distributions. However, with a large number of observational data, or a large parameter space, or in the combination of both, HMC can be quite computationally expensive. As expounded upon in Section 2, computing gradients of the log probability of the posterior becomes quite costly. As the number of leapfrog steps increases, the cost of producing one sample requires $L \cdot C(N, D, T)$, where L is

the number of leapfrog steps and $C(N, D, T)$ is the cost of auto-differentiating N samples over a parameter space of size D , in which the log probability function requires T operations. With complicated models such as Gaussian Processes, $C(N, D, T)$ can be polynomial with respect to N . Thus, one avenue to speed up HMC in practice is by attempting to decrease $C(N, D, T)$. Several methods for doing so are discussed in the next section.

1.2. Related Work

Much work has been devoted to developing “surrogate” methods to speed up HMC. For example, (Rasmussen, 2003) develop a Gaussian Process to learn the potential function during HMC burn-in steps. Instead of modeling the potential, (Li et al., 2019) utilize Neural Networks to estimate the gradient of the potential function instead. More recently, (Dhulipala et al., 2023) proposed taking advantage of Hamiltonian Neural Networks (HNNs) in order to approximate the gradient of the trajectories through phase space.

1.3. Contribution

While the aforementioned surrogate methods are sufficient, they do not explicitly constrain the solution space of their approximations to produce Hamiltonian trajectories that are consistent with Hamiltonian dynamics; training is conducted in such a way that learning the gradient or potential function is independent of the learned approximation’s effect on the resultant trajectory. Furthermore, the approximations are not necessarily constrained to preserve important qualities of the Hamiltonian.

Our contribution is novel, in that we propose a surrogate model training scheme which explicitly preserves the Hamiltonian properties. We show that in doing so, we require less training samples, and provide higher quality approximations. Furthermore, our method can be easily adapted to learning non-separable Hamiltonian dynamics, an application that is useful for reducing the computational complexity of sampling using RMHMC.

2. Background

2.1. Hamiltonian Monte Carlo

The Hamiltonian, $H(q, p)$, is a function of q and p , parameters defining an object's position and velocity. Flow through a Hamiltonian system can be defined as a symplectic map. We will discuss this formalism later. We can decompose the Hamiltonian into the following, if the Hamiltonian is separable:

$$H(q, p) = U(q) + K(p)$$

$U(q)$ and $K(p)$ represent potential and kinetic energy of the system respectively.

The dynamics of the system can be modeled by noting that

$$\begin{aligned}\frac{\partial H}{\partial q} &= -\frac{\partial p}{\partial t} \\ \frac{\partial H}{\partial p} &= \frac{\partial q}{\partial t}\end{aligned}$$

Thus, the system can be evolved over time using numerical integration schemes.

A Hamiltonian system has key properties:

1. Reversibility
2. Volume Preservation
3. Conservation of the Hamiltonian

Hamiltonian Monte Carlo (Brooks et al., 2011) takes advantage of this formulation by defining $U(q)$ as the negative log probability of the target distribution we wish to sample from, and $K(p) = \frac{p^T p}{2}$, a function of a kinetic parameter p sampled from a standard multivariate normal, with dimension equal to the dimensionality of the parameter space. Samples are produced by numerically evolving the system for some determined time-steps L using a step-size ϵ , and taking the final state of the system. A new momentum parameter is sampled, and the process is repeated.

Extensions including Riemannian Manifold Hamiltonian Monte Carlo (Betancourt, 2013) typically parametrize the kinetic energy $K(q, p) = \frac{p^T M(q)^{-1} p}{2}$, where $M(q)$ is taken to be the positive-definite Fisher Information matrix.

2.2. Leapfrog Integrators

An analytical form for the Hamiltonian dynamics is typically unknown, so discretization techniques are used to simulate Hamiltonian dynamics. In practice, a certain form of numerical integration scheme, called leapfrog integrators are used. These integrators have desirable properties, in that

they are time-reversible as well as conserve the Hamiltonian. Assuming a separable Hamiltonian form commonly seen in Hamiltonian Monte Carlo, $H(q, p) = U(q) + \frac{p^T p}{2}$, one may use the following update rules for a given starting point in the phase space (q_t, p_t) to simulate dynamics with a fixed step size ϵ ²:

$$\begin{aligned}q_{t+1} &= q_t + \epsilon p_t + \frac{\epsilon^2}{2} \frac{\partial p_t}{\partial t} \\ p_{t+1} &= p_t + \frac{\epsilon}{2} \left(\frac{\partial p_t}{\partial t} + \frac{\partial p_{t+1}}{\partial t} \right)\end{aligned}$$

Given a separable Hamiltonian in the HMC setting, we recall that $\frac{\partial p_t}{\partial t} = -\frac{\partial H}{\partial q_t}$. Thus, evolving a separable Hamiltonian system in a HMC setting requires the computation of $\frac{\partial U(q_t)}{\partial q_t}$. Typically, this is carried out using auto-differentiation techniques. However, for complicated potential functions $U(q)$, this computation can be quite costly (Li et al., 2019) if analytical forms are unknown.

2.3. Neural ODE

Neural ODEs (Chen et al., 2019) can be used to model continuous time dynamics. Note that if we aim to approximate a smooth function, $y(x) = f(x, \theta)$, then it is also equivalent to approximate $y(x) = \int \frac{\partial f(x, \theta)}{\partial x} dx$.

Neural ODEs aim to model $\frac{\partial f(x, \theta)}{\partial x}$ instead of traditionally modeling $f(x, \theta)$, and then integrate using an ODE solver to obtain an estimate for $y(x)$. The parameters θ are learned by efficiently backpropagating through the ODE solver dynamics. A variety of implementations, such as the torchdyn package (Poli et al., 2020) are available.

2.4. Symplectic Neural Networks

Symplectic Neural Networks (SNNs) (Jin et al., 2020) are a class of function approximators that can approximate arbitrary symplectic maps, under appropriate choices of activation functions and non-linear layers. The learned function $\hat{\phi}(z, t)$ can be thought of as a solution to flowing the initial condition z along the learned vector field after time t .

3. Methodology

3.1. Neural Network Gradient Hamiltonian Monte Carlo (NNgHMC)

Let $f(\theta, q)$ be a feed-forward neural network, parametrized by learnable parameters θ . Neural Network Gradient Hamiltonian Monte Carlo seeks to use $f(\theta, q)$ to approximate $\frac{\partial H}{\partial q}$ from burn-in trajectories on each leapfrog step during

²Equivalent symplectic integrators also exist for non-separable Hamiltonian forms, such as those seen in RMHMC (Tao, 2016).

Hamiltonian Monte Carlo. This amounts to learning the gradient of the negative log probability of the parameters q as a function of q . According to (Li et al., 2019), the pair of $(q, \frac{\partial H}{\partial q})$ are passed as training input and output for the aforementioned neural network.

3.2. Trajectory Guided Neural ODE Hamiltonian Monte Carlo

Let $f(\theta, q, p)$ be a Neural Network approximating the derivatives of the Hamiltonian of a system, $\frac{\partial H}{\partial q}$ and $\frac{\partial H}{\partial p}$, or equivalently $-\frac{\partial p}{\partial t}$ and $\frac{\partial q}{\partial t}$. Evolving this system forward using a leapfrog integration scheme is equivalent to modeling Hamiltonian dynamics, if the proposed approximators are accurate.

In order to train this network, we take training samples to be leapfrog trajectories themselves. That is, for a burn in period of K samples, using L leapfrog steps, our training input can be defined by $\{(q_i^0, p_i^0)\}_{i=1}^K$, the set of initial conditions for each of our Hamiltonian Monte Carlo trajectories. Our training target, however, can be defined as $\{(Q_i, P_i)\}_{i=1}^K$, where Q_i and P_i are vectors of length L corresponding to an L length trajectory for burn-in sample i .

The model is trained by providing initial conditions to $f(\cdot)$, evolving the system according to Hamiltonian dynamics using a Leapfrog integrator, computing the element-wise trajectory loss between generated and actual Hamiltonian trajectory as well as the gradient loss, and efficiently back-propagating through the ODE dynamics.

While both this approach as well as the NNgHMC both approximate the gradient of the potential function, the NNgHMC does not explicitly constrain the approximation to adhere to Hamiltonian dynamics. However, this approach implicitly conditions the approximation of the gradient to follow Hamiltonian dynamics by penalizing deviations from the observed trajectory.

Additionally, this method can be easily extended to Riemannian Manifold Hamiltonian Monte Carlo, by simply substituting Riemannian Manifold Hamiltonian Monte Carlo trajectories during the training period instead of the Hamiltonian Monte Carlo trajectories and modifying the leapfrog dynamics to handle non-separable Hamiltonians (Tao, 2016).

3.2.1. MODEL ARCHITECTURE AND OPTIMIZATION

Each model takes an input of size $2 \cdot D$, as the concatenated position q and momentum p are fed to the model, and consists of a single hidden layer of $100 \cdot D$ units, where D is the dimensionality of the parameter space. The **tanh** activation function is used to add non-linearity in the hidden layer. Brief descriptions of various architectures used are described below, along with accompanying diagrams 1. All

methods were implemented in pytorch (Paszke et al., 2019).

Models are trained for at most 100 epochs with the Adam optimizer at an initial learning rate of 10^{-2} . To keep the comparison between surrogates a comparison of *models* rather than of optimizer tuning, every surrogate in this work — including the NNgHMC baseline and the symplectic networks of Section 3.3.1 — is fit with an identical optimization protocol: the learning rate is halved on plateau (patience 6), training stops early if the objective fails to improve for 25 epochs, and the parameters attaining the lowest objective are restored at the end rather than those of the final epoch.

The training signal is the pair (trajectory loss, gradient loss). Warm-up trajectories are recorded at the $L + 1$ times $0, \epsilon, \dots, L\epsilon$, with the initial state included and the momentum reported at integer times: the leapfrog half-step is undone at *every* step, not only the last, so that each stored (q, p) pair is synchronized and the recorded states lie on the trajectory the surrogate is asked to reproduce.

3.2.2. APPROXIMATING HMC

We consider two model architectures, Neural ODE Gradient HMC (NNODEgHMC) and Hamiltonian Neural ODE Gradient HMC (HNNODEgHMC). The former produces f which approximates $\frac{\partial H}{\partial p}$ and $\frac{\partial H}{\partial q}$ directly, while the latter contains a sub-model f which approximates $H(q, p)$. Through gradient enabled auto-differentiation of this sub-model, $\frac{\partial H}{\partial p}$ and $\frac{\partial H}{\partial q}$ are produced. The resulting gradient approximations are evolved forward using leapfrog steps.

3.2.3. APPROXIMATING RMHMC

We consider two model architectures, Neural ODE Gradient RMHMC (NNODEgRMHMC) and Hamiltonian Neural ODE Gradient RMHMC (HNNODEgRMHMC). Much like the standard HMC setting, the former produces f which approximates $\frac{\partial H}{\partial p}$ and $\frac{\partial H}{\partial q}$ directly, while the latter contains a sub-model f which approximates $H(q, p)$. Through gradient enabled auto-differentiation of this sub-model, $\frac{\partial H}{\partial p}$ and $\frac{\partial H}{\partial q}$ are produced. A subtle difference here is that our Hamiltonian is non-separable, so a symplectic integrator as in (Tao, 2016) is used to evolve the system forward.

3.3. Symplectic Neural Network Hamiltonian Monte Carlo

Denoting $z = (q, p)$, the learned function $\hat{\phi}(z, t)$ is symplectic, and can be thought of as a solution to finding the exponential map of the Hamiltonian vector field. Since Hamiltonian flow coincides with the geodesic in phase space, $\hat{\phi}(z, t)$ can be used to parametrize such geodesics if trained on Hamiltonian trajectories, given initial conditions z .

We implement two architectures proposed by (Jin et al.,

2020), **LA-Symplectic-Net** and **G-Symplectic-Net**. Details about the implementation can be found in the aforementioned paper.

Our training data involves generating all possible pairs of start and end points within the evolution of each trajectory. For example, given a trajectory of length L , we generate the training pairs $\{(x_i, x_j)\}_{i=1, j=1}^L$ where $i < j$. Here, x_i is the position in phase space at time $t = i$. We seek to minimize the loss between $\hat{\phi}(x_i, j - i)$ and x_j .

Gradient information can in fact be supplied, and we do so here. Because $\hat{\phi}(z, \tau)$ is the time- τ flow map, its generator is recovered by differentiating in τ at the origin,

$$d\hat{\phi}(z, \tau) \tau \Big|_{\tau=0} = (dH p, -dH q)(z), \quad (1)$$

so the vector field the SNN implies at any observed state is a single directional derivative of the network with respect to its time argument. We evaluate (1) in forward mode: τ is a scalar, so one Jacobian-vector product yields the whole $2D$ -dimensional velocity, whereas reverse mode would require a backward pass per output component. Matching the momentum half of (1) to the recorded $\nabla \log p$ supplies the same gradient supervision the integrator-based surrogates receive; the position half must equal p , which the trajectory already provides at no extra cost. For non-separable (RMHMC) trajectories we record the complete field $(\partial_p H, -\partial_q H)$ during warm-up — it is already formed at every stage of the integrator — and match both halves.

3.3.1. MODEL ARCHITECTURE AND OPTIMIZATION

Each model takes an input of size $2 \cdot D$, as the concatenated position q and momentum p are fed to the model, as well as the time evolution vector of length L denoting the step size taken at time t . The **Swish** (Ramachandran et al., 2017) activation function is used to add non-linearity in the hidden layers, as the architecture of SNNs requires deep composition of layers. All methods were implemented in pytorch (Paszke et al., 2019).

Depth. Both architectures use 8 alternating (up/down) blocks: the G-SympNet with width $100 \cdot D$ per block, the LA-SympNet with 8 channels per linear block. Depth is not a free tuning choice here. Each block is a shear, and a composition of two shears cannot represent a rotation: for the linear map $[[I + BA, B], [A, I]]$ obtained from two shears, matching a rotation through angle ϑ requires simultaneously $1 + BA = \cos \vartheta$ and $BA = -\sin^2 \vartheta$, which forces $\cos^2 \vartheta = \cos \vartheta$ and so admits only $\vartheta \in \{0, \pi/2\}$. Since the exact flow of a Gaussian target is a rotation in each (q_i, p_i) plane through angle $\omega_i t$, a two-block network cannot represent it at any width — leapfrog itself requires three shears. Empirically, fitting a noiseless rotation of 3 radians attains a mean-squared error of 1.0 with two blocks

and 2×10^{-4} with eight.

Conditioning. A shear scaled by τ amplifies the state by approximately $(1 + \|A\|\tau)$, so a composition of n shears with unit-norm initialization amplifies by $(1 + \tau)^n$. For the default 64 shears of the LA architecture this reaches $\sim 10^{12}$ at $\tau = 1.5$, which is within the range of time offsets seen by any target whose step size is not small. We therefore rescale the shear weights by $1/n$ at construction, which returns the amplification at $\tau = 1.5$ to 1.04 and leaves the closed-form inverses of Section 3.4 exact to machine precision.

Optimization. SNNs are trained for at most 300 epochs in mini-batches of 4096 pairs, under the same protocol as Section 3.2.1: Adam at 10^{-2} , learning rate halved on plateau, early stopping (patience 20), and restoration of the best iterate. The number of training pairs grows quadratically in L , so mini-batching rather than full-batch descent is what makes the trajectory-pair construction tractable. Because the gradient term of (1) is typically one to two orders of magnitude larger than the trajectory term at initialization, we rescale it once, at the start of training, by the ratio of the two losses; without this the gradient term dominates the objective that actually determines proposal quality, and the ablation of Figure 9 would compare loss weightings rather than supervision signals.

3.3.2. APPROXIMATING HMC / RMHMC

Since SNNs can approximate any arbitrary symplectic map, there is no tangible difference between the training procedures of dealing with HMC and RMHMC trajectories. We consider the two model architectures previously mentioned, **LA-Symplectic-Net** and **G-Symplectic-Net**.

3.4. Exactly Reversible Symplectic Network Proposals

Symplectic Neural Networks are symplectic by construction, and therefore volume preserving. We show that this is *not* sufficient for them to serve as a valid Hamiltonian Monte Carlo proposal, and give a construction that repairs the deficiency exactly, at no cost in parameters.

3.4.1. VALIDITY CONDITIONS FOR A DETERMINISTIC PROPOSAL

Let $z = (q, p) \in \mathbb{R}^{2D}$ and let $\pi(z) \propto e^{-H(z)}$ be the target on phase space. Write $R(q, p) = (q, -p)$ for the momentum flip, so that $R^2 = \text{Id}$, and assume as usual that the Hamiltonian is even in the momentum, $H \circ R = H$. A Metropolis–Hastings scheme that proposes $z' = T(z)$ deterministically and accepts with probability $\min\{1, e^{H(z)-H(Tz)}\}$ leaves

π invariant provided

$$(V) \quad |\det \nabla T(z)| = 1 \quad (\text{volume preservation}), \quad (2)$$

$$(I) \quad (R \circ T) \circ (R \circ T) = \text{Id} \quad (\text{involutivity}). \quad (3)$$

Condition (2) removes the Jacobian factor from the acceptance ratio and (3) supplies the reverse move required by detailed balance; see (Green, 1995; Tierney, 1998). The leapfrog integrator satisfies both for *any* force field depending on position alone, which is why classical surrogate methods that learn ∇U and integrate it remain exact: an inaccurate network lowers the acceptance rate but never biases the chain.

A learned flow map is a different situation. An SNN Φ_t satisfies (2) because each of its layers is a shear, but nothing in its architecture or training objective enforces (3). Empirically the involution defect $\|(R\Phi_t)^2 z - z\|_\infty$ is of order 10^{-1} both at initialisation and after training, so the resulting chain does not target π , and the discrepancy does not vanish as the training set grows unless Φ_t converges to the exact flow.

3.4.2. THE SYMMETRIZED MAP

Every SNN layer updates either q or p using a function of the *other*, unmodified, half of the state. Such a shear is invertible in closed form: the increment can be recomputed from the output and subtracted. Composing these layer inverses in reverse order gives an exact inverse Φ_t^{-1} , evaluated at the same cost as Φ_t and to machine precision (we measure $\|\Phi_t^{-1}\Phi_t z - z\|_\infty \approx 10^{-15}$).

Given this, define the *time-symmetric* map

$$\Psi_t := R \circ \Phi_{t/2}^{-1} \circ R \circ \Phi_{t/2}. \quad (4)$$

Proposition 3.1. *For any $\Phi_{t/2}$ that is invertible, Ψ_t satisfies (3). If in addition $\Phi_{t/2}$ is symplectic, Ψ_t satisfies (2). Consequently the Metropolis–Hastings chain with proposal Ψ_t and acceptance $\min\{1, e^{H(z)-H(\Psi_t z)}\}$ leaves π invariant, irrespective of the accuracy of Φ .*

Proof. Write $\Phi := \Phi_{t/2}$. Using $R^2 = \text{Id}$ in (4),

$$R \circ \Psi_t = R \circ R \circ \Phi^{-1} \circ R \circ \Phi = \Phi^{-1} \circ R \circ \Phi.$$

This is a conjugation of the involution R , hence an involution:

$$(R \circ \Psi_t)^2 = \Phi^{-1} R \Phi \Phi^{-1} R \Phi = \Phi^{-1} R^2 \Phi = \Phi^{-1} \Phi = \text{Id},$$

which is (3). For (2), note R is anti-symplectic: it reverses the sign of the canonical form, $R^*(dq \wedge dp) = -dq \wedge dp$. Therefore $R \circ \Phi^{-1} \circ R$ is symplectic, being a composition of two anti-symplectic maps with a symplectic one, and Ψ_t

is a composition of two symplectic maps, hence symplectic. Symplecticity implies $|\det \nabla \Psi_t| = 1$. Neither argument uses any property of Φ beyond invertibility and symplecticity, so both hold for arbitrary network weights. $\square$

The half-step in (4) makes the construction *consistent*: it does not perturb an already-exact model.

Lemma 3.2. *If Φ_s is the exact time- s flow of a Hamiltonian H with $H \circ R = H$, then $\Psi_t = \Phi_t$.*

Proof. Reversibility of such a flow gives $R \circ \Phi_s \circ R = \Phi_{-s} = \Phi_s^{-1}$, hence $\Phi_{t/2}^{-1} = R \circ \Phi_{t/2} \circ R$. Substituting into (4),

$$\Psi_t = R (R \Phi_{t/2} R) R \Phi_{t/2} = \Phi_{t/2} \circ \Phi_{t/2} = \Phi_t. \quad \square$$

Equation (4) is the classical adjoint (symmetrisation) composition of geometric integration (Hairer et al., 2006), in which a method is composed with its adjoint $\Phi_t^* = \Phi_{-t}^{-1}$ to obtain a symmetric method; here the adjoint is realised as $R \circ \Phi^{-1} \circ R$. The same construction is used by (Valperga et al., 2022) to build reversible symplectic architectures for learning chaotic Hamiltonian dynamics, where it is applied to Hénon maps and coupling layers rather than to SympNet modules and is not connected to sampling. Enforcing exact invertibility so that a learned proposal admits a tractable reverse move is likewise established in the neural-MCMC literature (Song et al., 2017; Levy et al., 2018), and (Spanbauer et al., 2020) give a general construction from volume-preserving involutions. What we add is the observation that SNN surrogates for HMC require this structure — they supply (2) but violate (3), so the resulting chain is not asymptotically exact — together with closed-form layer inverses that make (4) exact for the LA- and G-SympNet architectures.

3.4.3. INTERPRETATION AND COST

Proposition 3.1 should be read as a statement about *structure*, not accuracy. The learned map is not thereby made a better approximation of the Hamiltonian flow; rather, Ψ_t is constrained to possess a symmetry the true flow already possesses, namely $R\Psi_t R = \Psi_t^{-1}$. In this sense the symmetrisation plays the same role for time-reversal that the SympNet architecture plays for the symplectic form: it restricts the hypothesis class to maps that the exact solution inhabits. Since the constraint is satisfied identically in the weights, it also removes the need to spend training signal on a property that can be imposed for free.

The map (4) evaluates the network twice, each at half the step, and introduces *no additional parameters*. Note that a Ψ built from an n -block network has the same effective depth as a plain $2n$ -block network while carrying half as

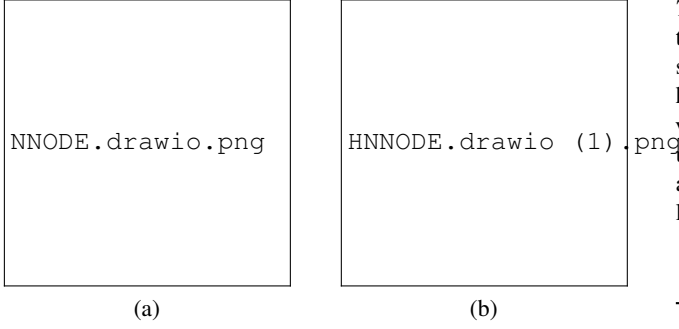


Figure 1. Neural Network Architectures used in experiments. Note (a) outputs the gradients directly, while (b) first models the Hamiltonian and then produces the gradients via auto-differentiation.

many parameters; comparisons should therefore control for effective depth rather than block count.

Finally, the property is preserved under composition, so trajectories built from repeated application remain valid proposals: if $R\Psi R = \Psi^{-1}$ then $R\Psi^k R = (R\Psi R)^k = \Psi^{-k}$, whence $(R\Psi^k)^2 = \text{Id}$ for every $k \in \mathbb{N}$.

4. Experiments

We evaluate our method against baseline “surrogate” and traditional methods by several criteria:

1. Effective Samples per second
2. Adherence to Hamiltonian properties (conservation, reversibility, symplecticness)

4.1. Synthetic Results

In the following plots, each model was trained on the warm-up trajectories generated by HMC. The table below details the parameters of the experiment (step size, samples), as well as the posterior distribution, up to a constant. The number of posterior samples generated was always equal to the number of training samples. All trajectories used $L = 5$ leapfrog steps and are recorded at the $L + 1 = 6$ times $0, \epsilon, \dots, 5\epsilon$, the initial state included; the surrogates are consequently supervised on time offsets up to 5ϵ , which is also the horizon at which they are asked to propose.

5. Discussion and Future Work

Experiments in the Hamiltonian Monte Carlo setting for a few distributions showed that adding a trajectory deviation penalty to traditional NNgHMC (Li et al., 2019) allows for more efficient training; as shown in Figure 9, fewer training samples were needed to produce proposals of similar quality. However, as the NNODEgHMC models are trained

Table 1. Experiment parameters, including from L to R distribution name, log posterior density, step size, and number of training samples. The GP regression target is the posterior over three hyperparameters (log lengthscale, log signal variance, log noise variance) of a squared-exponential GP fit to 200 observations under standard normal priors; each gradient evaluation requires an $O(n^3)$ Cholesky factorization, placing it in the expensive-likelihood regime that motivates surrogate methods.

DISTRIBUTION	$\log(p(\theta y))$	η	SAMPLES
BANANA	$-\frac{\theta_1^2}{200} - \frac{(\theta_2 + \theta_1^2 - 100)^2}{2}$	.1	$3 \cdot 10^3$
GAUSSIAN	$\log(\mathcal{N}(0, \text{diag}(.25, 1, 4)))$	.3	10^3
NORMAL-GAMMA	$-\frac{(\theta_1^2)}{2 \cdot 1.5} - \frac{(1.7 - \theta_1)^2}{2 \cdot \theta_2}$	.1	$3 \cdot 10^3$
HIGH-DIMENSIONAL GAUSSIAN	$-\frac{\sum_i^{30} (\theta_i)^2}{2}$	.1	$3 \cdot 10^3$
GP REGRESSION	$\log(\mathcal{N}(y; 0, K_\theta)) + \log(\mathcal{N}(\theta; 0, I))$	.05	$1.5 \cdot 10^3$

using Neural ODE methods, longer trajectory length could prove to be computationally costly. The benefit of gradient based models which are integrated with leapfrog integrators was clearly shown; they behave with proper Hamiltonian dynamics, if given enough training samples.

On the contrary, directly learning the flow of the Hamiltonian through SNNs proved to be an interesting experiment; SNNs were able to produce quality samples from the posterior while approximating Hamiltonian dynamics. Unlike the integrator-based surrogates, whose training cost is independent of trajectory length, the SNN objective grows quadratically in L through the pair construction, which mini-batching mitigates but does not remove.

Two limitations of the flow-learning approach are worth separating. The first is statistical: what the SNN must learn is a map on the full $2D$ -dimensional phase space, parameterized by time, whereas the gradient-based surrogates learn a field on the D -dimensional position space alone, from labels the warm-up phase already produces. The second is structural, and is the more consequential of the two: as shown in Section 3.4, an unmodified SNN supplies volume preservation but not involutivity, so it is not a valid Metropolis proposal at all, and its bias does not diminish with additional training data. The symmetrized construction Ψ_t repairs this exactly and at no cost in parameters, and it is the form in which we would recommend the architecture be used. We caution that Ψ_t evaluates the underlying network twice, so comparisons against unsymmetrized baselines must control for effective depth rather than block count.

Approximating RMHMC is a natural extension of this work, and one where the economics are more favorable: because Riemannian trajectories require a metric tensor — and hence second derivatives of the log density — at every stage of the integrator, the cost per exact trajectory is far higher, so a surrogate has correspondingly more to save. Two aspects transfer directly. Trajectories are generated with the explicit

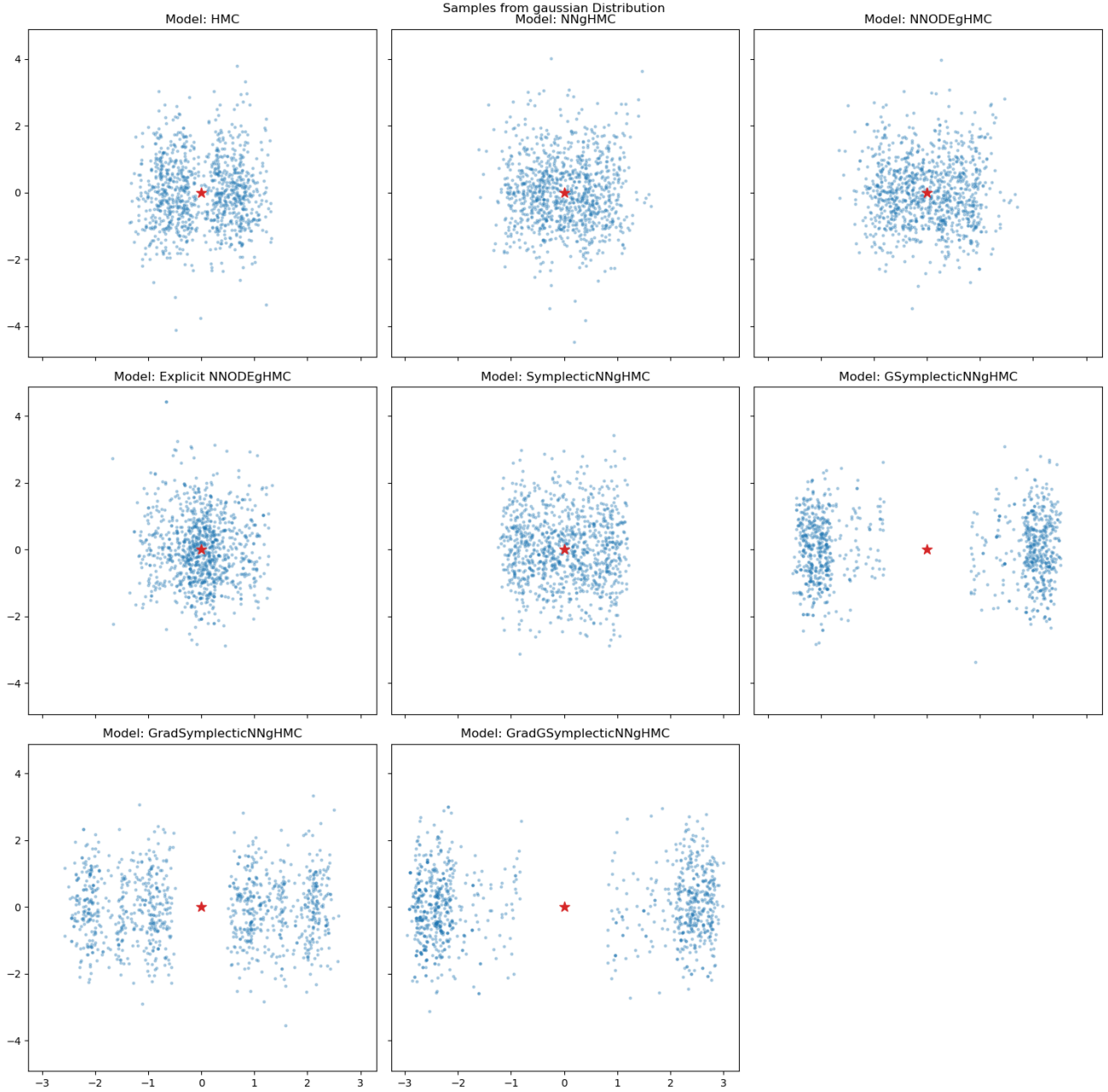


Figure 2. Samples from each method for the multivariate Gaussian. The integrator-based surrogates reproduce the target. The unsymmetrized symplectic networks (G-SympNet and both gradient-supervised variants) visibly do not: they place mass in two lobes with a hole at the mode, which is not a feature of the target but the bias of an invalid proposal. This is the failure of (3) made visible — these maps are *exactly* symplectic, so the defect is not approximate symplecticity but the missing involution (Section 3.4); the symmetrized construction (4) repairs it.

integrator of (Tao, 2016) on the augmented state, and the complete phase-space field $(\partial_p H, -\partial_q H)$ is recorded during warm-up at no meaningful additional cost, since the integrator already forms it; that field supplies gradient supervision for the non-separable case exactly as $\nabla \log p$ does in the separable one. Because SNNs approximate arbitrary

symplectic maps, the flow-learning architecture requires no modification whatsoever for the non-separable setting — only the source of the training trajectories changes.

Reversibility of gaussian

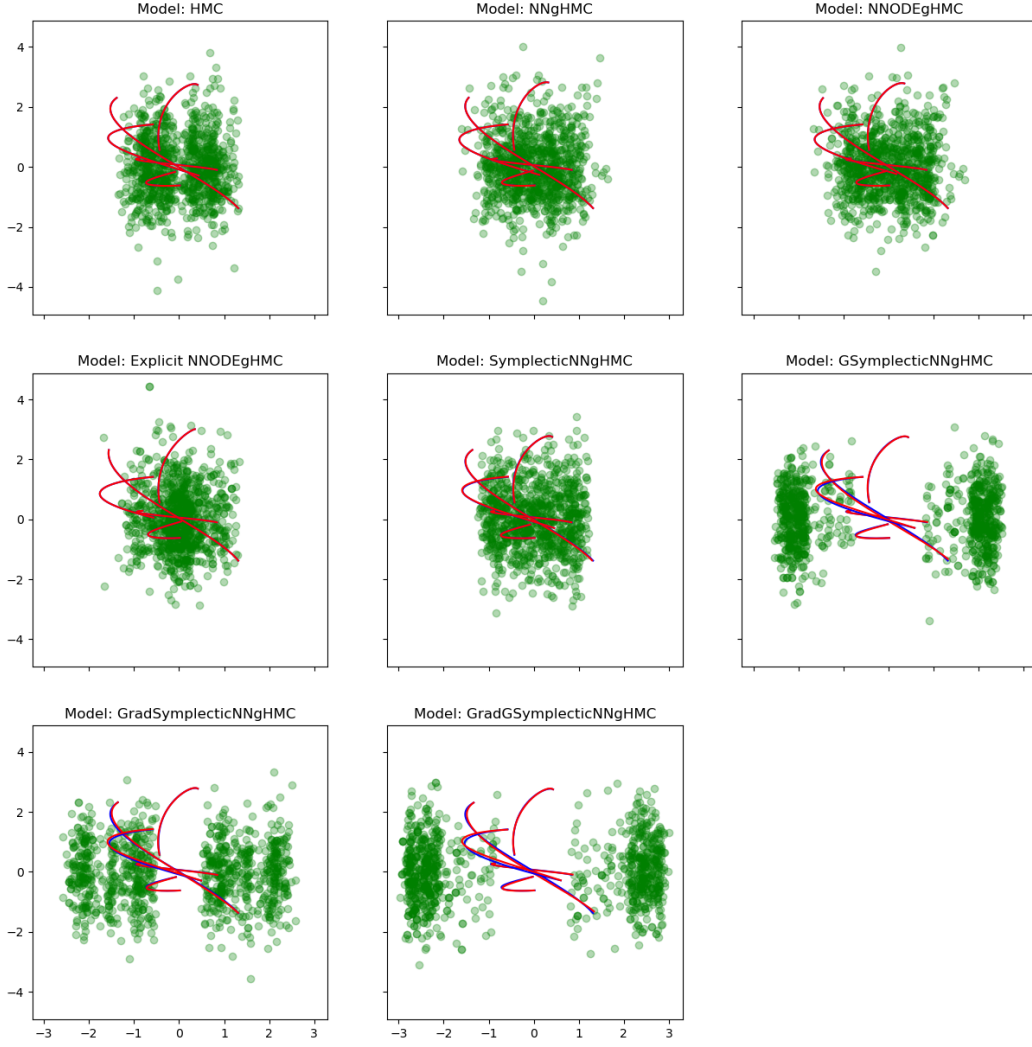


Figure 3. Reversibility of the Hamiltonian paths for the multivariate Gaussian: forward trajectories (blue) and the paths obtained by flipping the momentum and integrating back (red), over the sampled positions (green). Quantitative values for every target are given in Figure 10. Note that the unsymmetrized symplectic networks sample a split support, so their backward paths start from a region the target does not concentrate on.

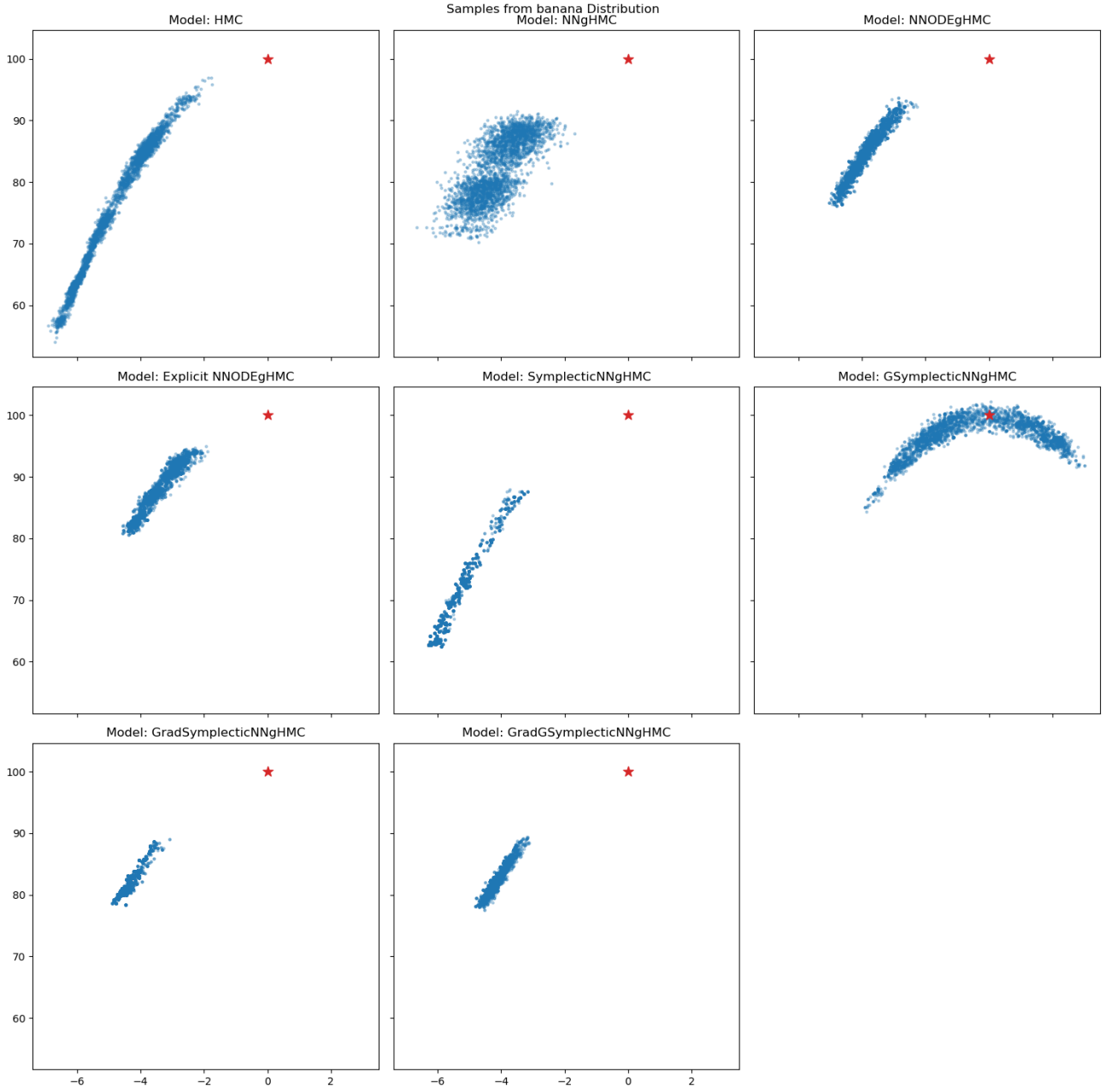


Figure 4. Samples from each method for the Banana distribution. This target should not be read as a comparison between methods: exact HMC itself attains an effective sample size of order 1 here at $L = 5$, $\epsilon = 0.1$, so no method is sampling it adequately. The banana’s curvature grows with θ_1 , which caps the stable step size while the marginal scale demands long trajectories — a conditioning problem that a fixed identity mass matrix cannot resolve, and one better addressed by the Riemannian methods of Section 3.4.

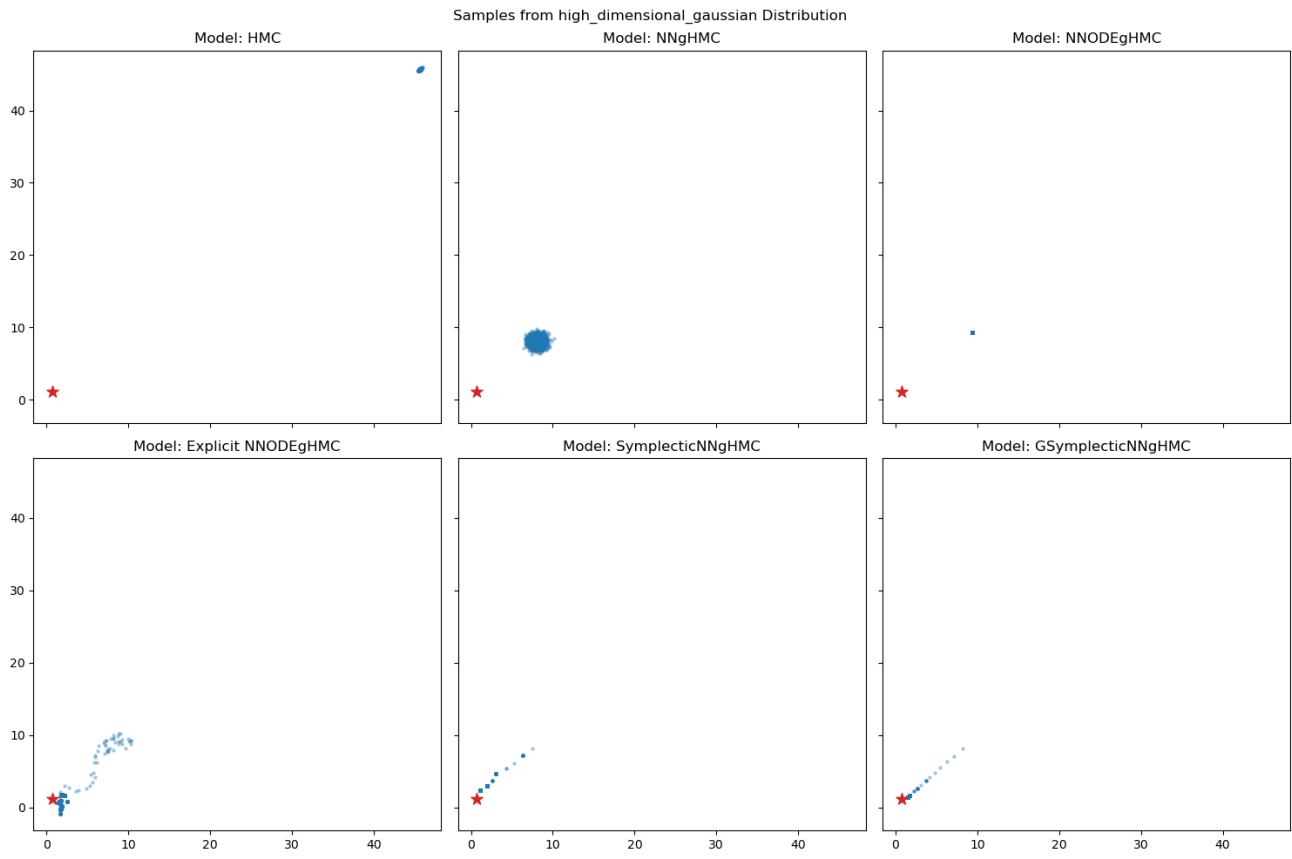


Figure 5. Samples from each method for the 30-dimensional Gaussian (first two coordinates shown).

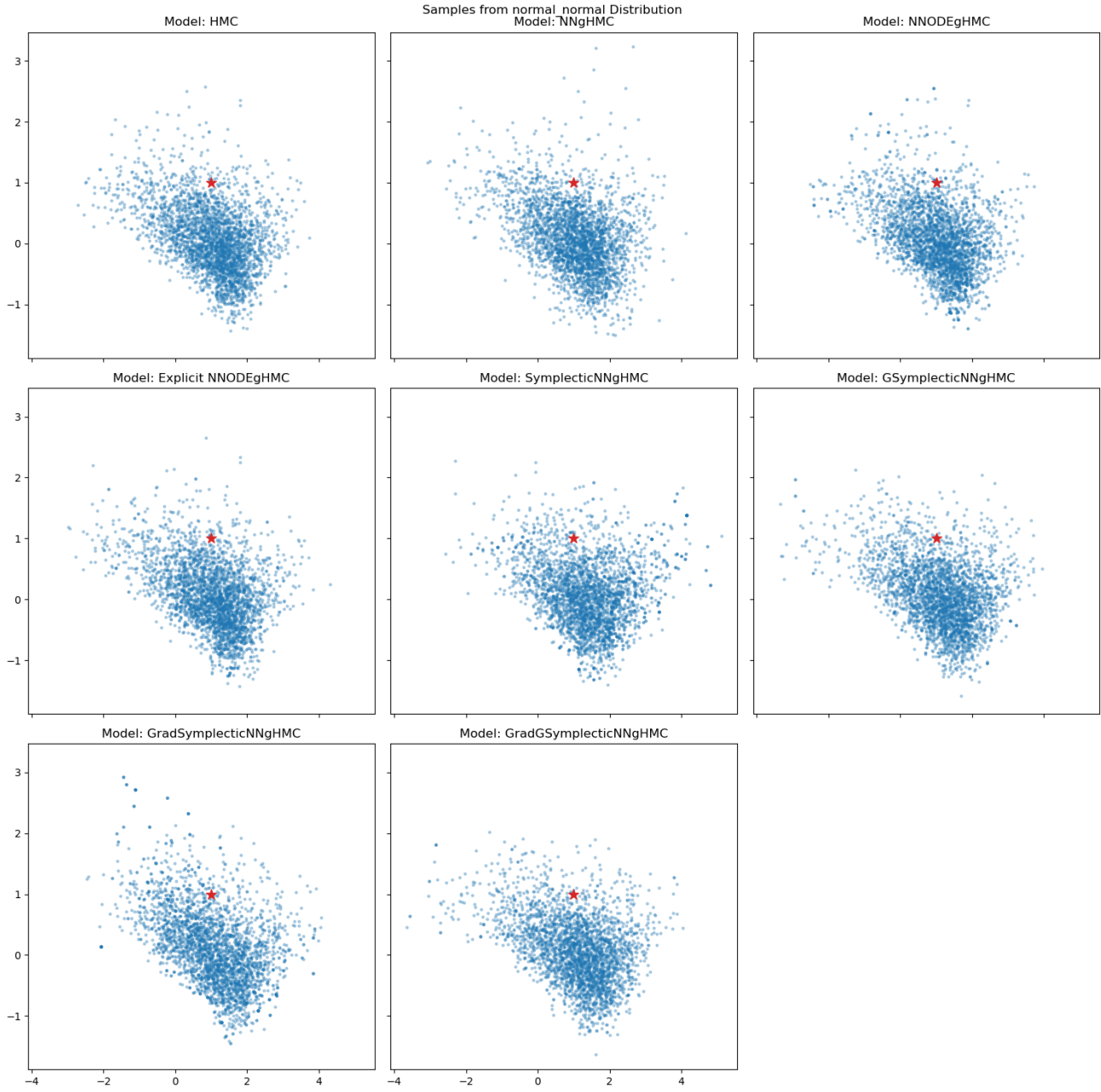


Figure 6. Samples from each method for the Normal-Gamma posterior.

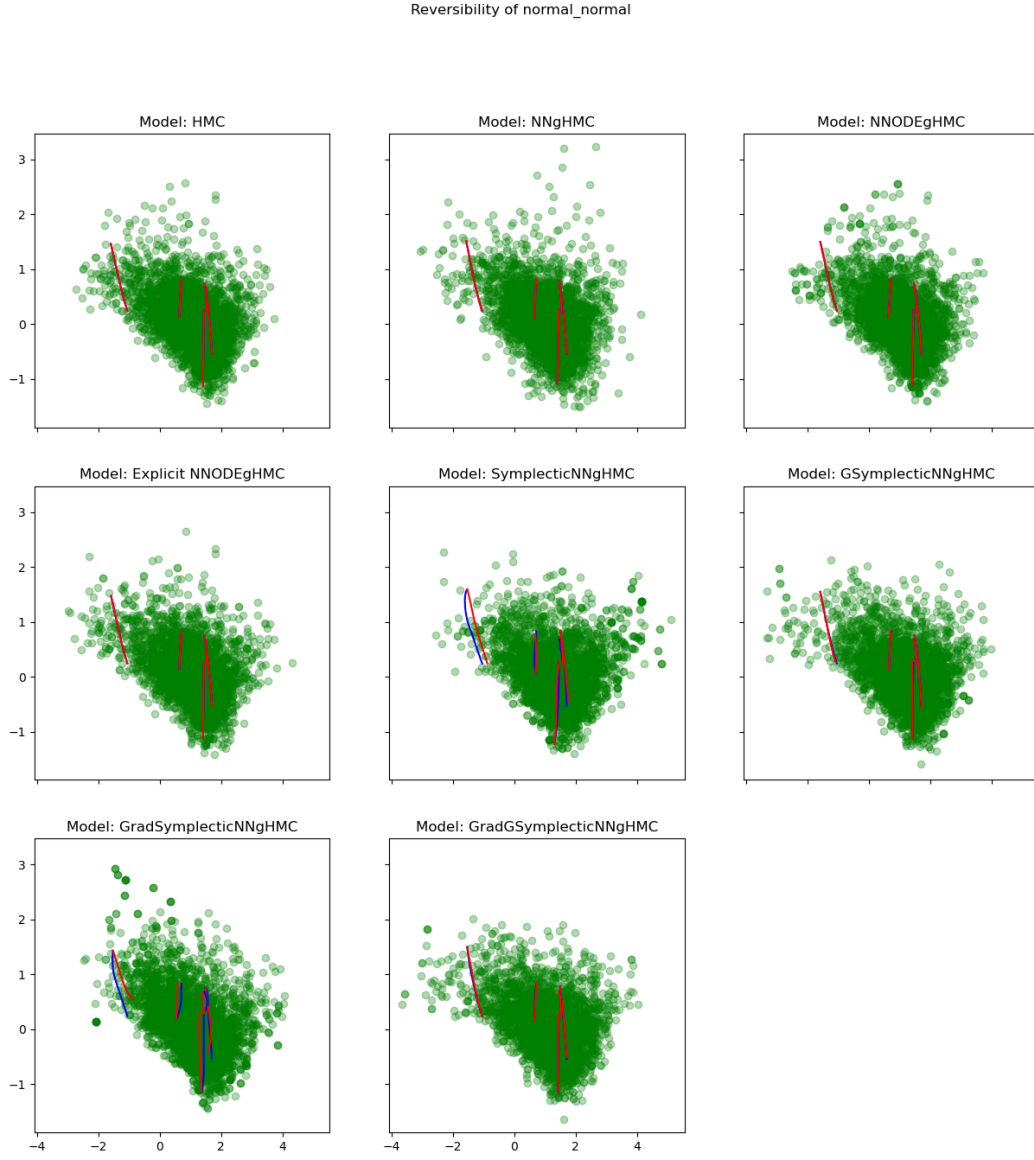


Figure 7. Reversibility of the Hamiltonian paths for the Normal-Gamma distribution: forward trajectories (blue) and the paths obtained by flipping the momentum and integrating back (red), over the sampled positions (green). Quantitative values are given in Figure 10.

Effective samples per second at the full training budget

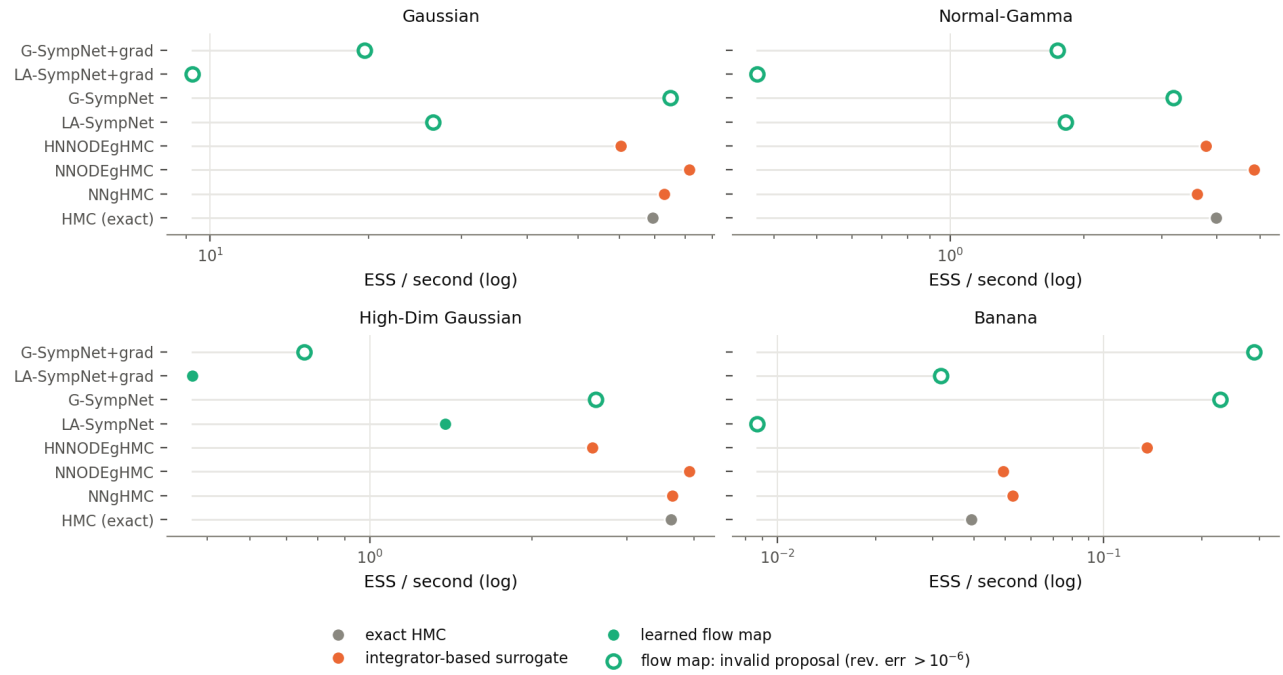


Figure 8. Effective samples per second at the full training budget (log scale; dot plots rather than bars, since a bar encodes magnitude by length from zero and a log axis has no zero). Markers are coloured by family; unsymmetrized flow maps whose reversibility error exceeds 10⁻⁶ are drawn as open circles, because their ESS does not certify a valid sampler. Among methods that are valid proposals, trajectory-guided NNODEgHMC attains the highest ESS/s on three of the four targets, modestly ahead of exact HMC (81.3 versus 69.3 on the Gaussian). The banana column should not be read as a comparison: exact HMC itself achieves an ESS of order 1 there, so no method is sampling that target adequately.



Figure 9. ESS/s as a function of the fraction of burn-in trajectories used for surrogate training, across all four distributions. NNODEgHMC (trajectory-guided) reaches competitive ESS/s with substantially fewer training samples than NNgHMC, supporting the claim that trajectory-level supervision enables more sample-efficient training. HMC is shown as a dashed reference. SNN methods are omitted from this panel: as Figure 10 shows, the unsymmetrized variants are not valid proposals, so their ESS/s is not interpretable as sampling efficiency.

Reversibility: integrator-based methods are exact by construction; unsymmetrized flow maps are not

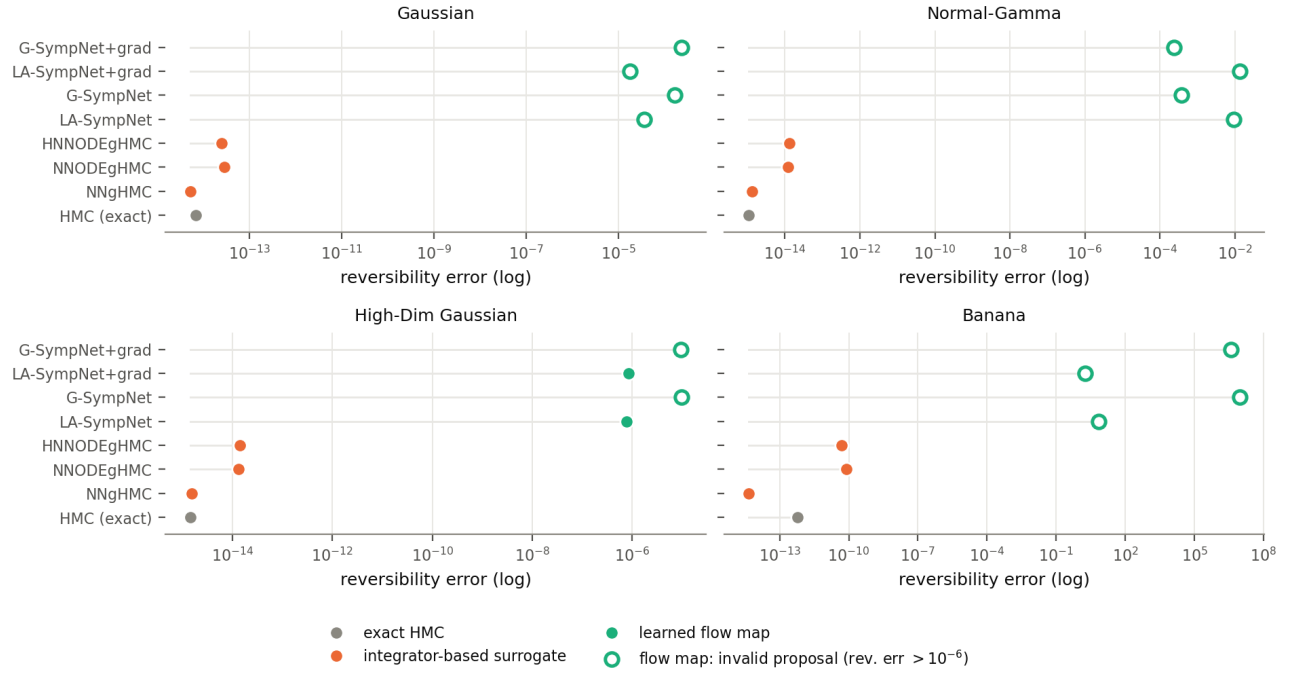


Figure 10. Reversibility error for each method, log scale. A Hamiltonian flow is time-reversal symmetric, so flipping the momentum and travelling for L steps must return the initial position; the error is the distance between the starting point and the point so reached. The integrator-based methods are exact to machine precision (1.3×10^{-15} to 7.7×10^{-11}), which is a property of the integrator and holds for any learned field. The unsymmetrized SNNs are not: their error spans 7.7×10^{-7} to 9.3×10^6 , i.e. up to thirteen orders of magnitude worse. This is a structural deficiency rather than an approximation error, and it does not diminish with more training data; see Section 3.4.

Hamiltonian conservation along proposal trajectories

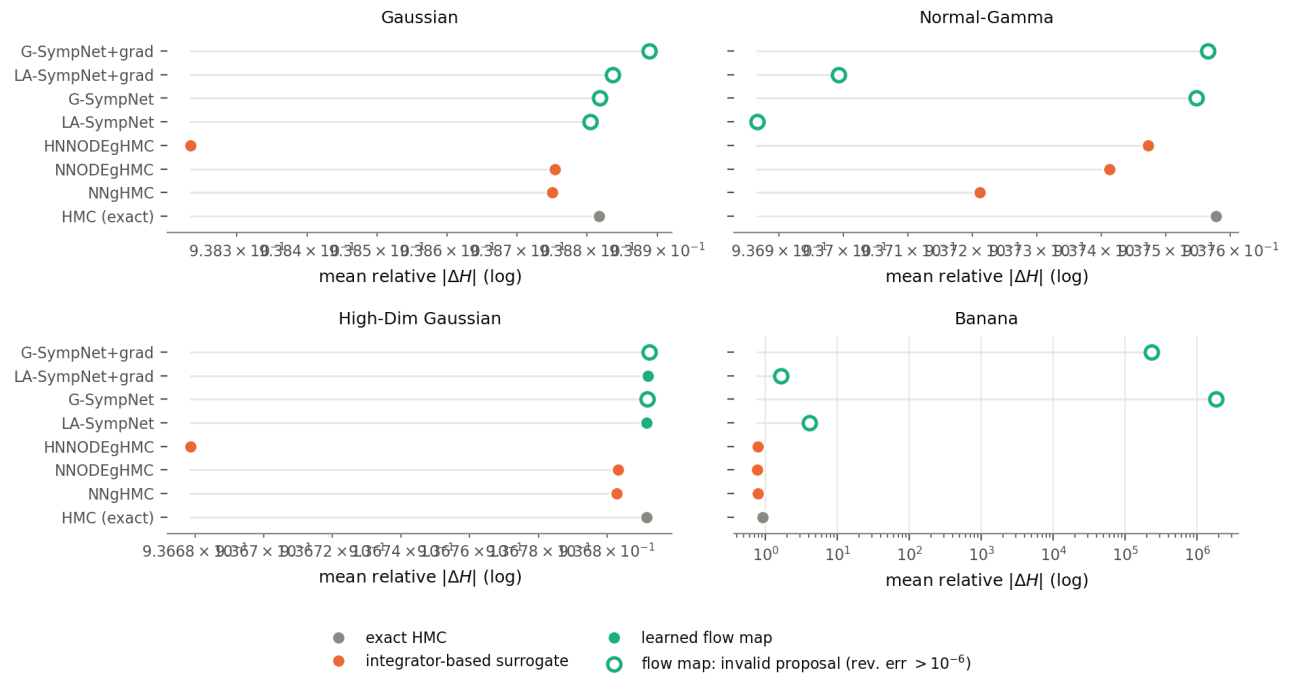


Figure 11. Mean relative change in the Hamiltonian along proposal trajectories, log scale. A symplectic integrator applied to a learned field conserves a modified Hamiltonian and so shows little systematic drift; the integrator-based surrogates behave accordingly. The unsymmetrized SNNs show substantially larger drift. We caution that conservation and reversibility are distinct properties: a map can be exactly symplectic, and hence volume preserving, while failing the involution condition that the Metropolis correction requires (Figure 10).

References

- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. Automatic differentiation in machine learning: a survey, 2018.
- Betancourt, M. *A General Metric for Riemannian Manifold Hamiltonian Monte Carlo*, pp. 327–334. Springer Berlin Heidelberg, 2013. ISBN 9783642400209. doi: 10.1007/978-3-642-40020-9_35. URL http://dx.doi.org/10.1007/978-3-642-40020-9_35.
- Brooks, S., Gelman, A., Jones, G., and Meng, X.-L. *Handbook of Markov Chain Monte Carlo*. Chapman and Hall/CRC, May 2011. ISBN 9780429138508. doi: 10.1201/b10905. URL <http://dx.doi.org/10.1201/b10905>.
- Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations, 2019.
- Dhulipala, S. L., Che, Y., and Shields, M. D. Efficient bayesian inference with latent hamiltonian neural networks in no-u-turn sampling. *Journal of Computational Physics*, 492:112425, 2023. ISSN 0021-9991. doi: <https://doi.org/10.1016/j.jcp.2023.112425>. URL <https://www.sciencedirect.com/science/article/pii/S002199912300520X>.
- Green, P. J. Reversible jump markov chain monte carlo computation and bayesian model determination. *Biometrika*, 82(4):711–732, 1995.
- Greydanus, S., Dzamba, M., and Yosinski, J. Hamiltonian neural networks, 2019.
- Hairer, E., Lubich, C., and Wanner, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, volume 31 of *Springer Series in Computational Mathematics*. Springer, 2 edition, 2006.
- Hoffman, M. D. and Gelman, A. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo, 2011.
- Jin, P., Zhang, Z., Zhu, A., Tang, Y., and Karniadakis, G. E. Sympnets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems, 2020.
- Levy, D., Hoffman, M. D., and Sohl-Dickstein, J. Generalizing hamiltonian monte carlo with neural networks, 2018.
- Li, L., Holbrook, A., Shahbaba, B., and Baldi, P. Neural network gradient hamiltonian monte carlo. *Computational Statistics*, 34(1):281–299, January 2019. ISSN 1613-9658. doi: 10.1007/s00180-018-00861-z. URL <http://dx.doi.org/10.1007/s00180-018-00861-z>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raiison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library, 2019.
- Poli, M., Massaroli, S., Yamashita, A., Asama, H., and Park, J. Torchdyn: A neural differential equations library, 2020.
- Ramachandran, P., Zoph, B., and Le, Q. V. Searching for activation functions, 2017.
- Rasmussen, C. E. Gaussian Processes to Speed up Hybrid Monte Carlo for Expensive Bayesian Integrals. In *Bayesian Statistics 7: Proceedings of the Seventh Valencia International Meeting*. Oxford University Press, 07 2003. ISBN 9780198526155. doi: 10.1093/oso/9780198526155.003.0045. URL <https://doi.org/10.1093/oso/9780198526155.003.0045>.
- Song, J., Zhao, S., and Ermon, S. A-nice-mc: Adversarial training for mcmc, 2017.
- Spanbauer, S., Freer, C., and Mansinghka, V. Deep involutive generative models for neural mcmc, 2020.
- Tao, M. Explicit symplectic approximation of nonseparable hamiltonians: Algorithm and long time performance. *Physical Review E*, 94(4), October 2016. ISSN 2470-0053. doi: 10.1103/physreve.94.043303. URL <http://dx.doi.org/10.1103/PhysRevE.94.043303>.
- Tierney, L. A note on metropolis-hastings kernels for general state spaces. *The Annals of Applied Probability*, 8(1): 1–9, 1998.
- Valperga, R., Webster, K., Turaev, D., Klein, V., and Lamb, J. Learning reversible symplectic dynamics. In *Proceedings of The 4th Annual Learning for Dynamics and Control Conference*, volume 168 of *Proceedings of Machine Learning Research*, pp. 906–916. PMLR, 2022.